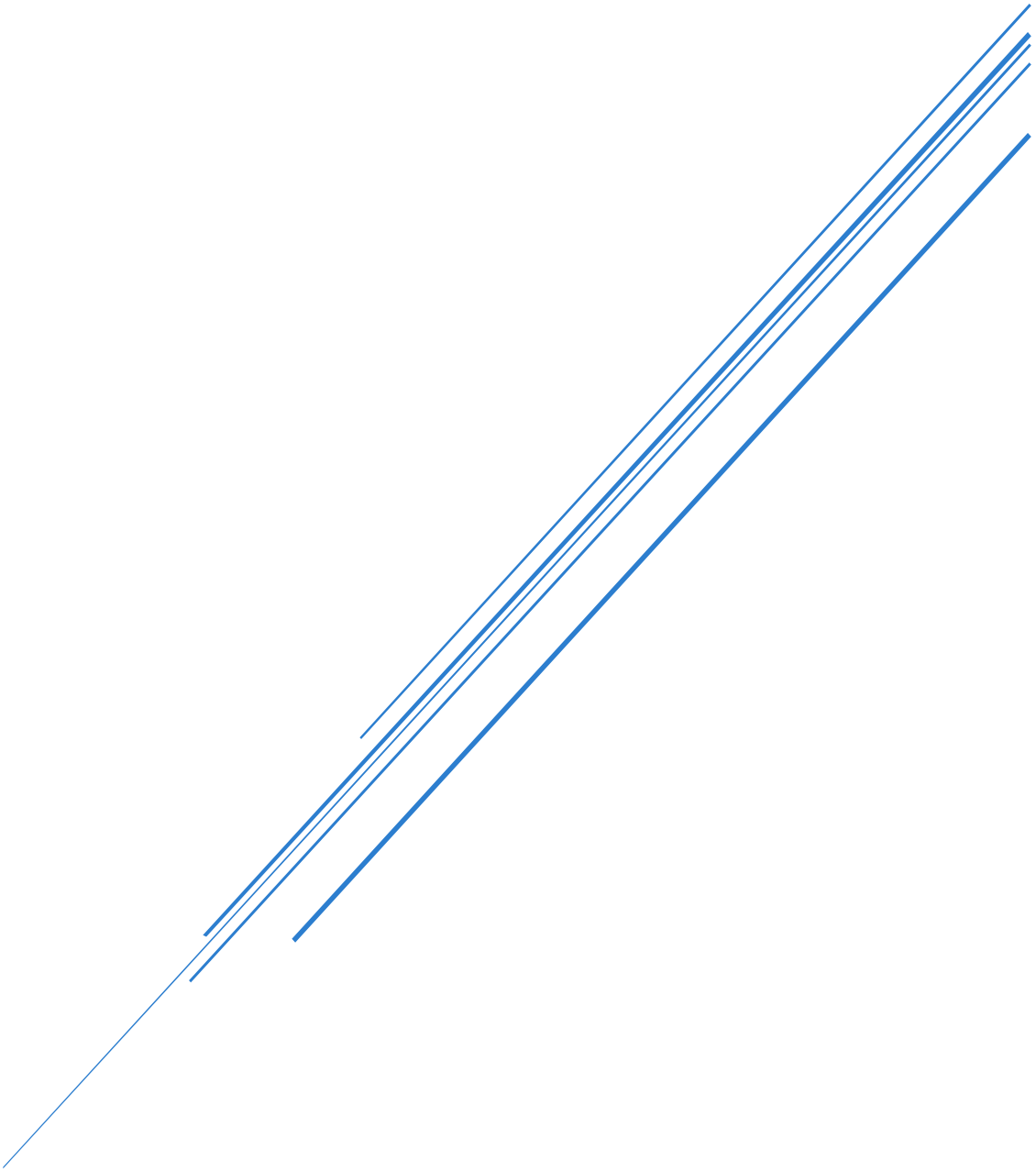
ARQUITECTURA DE COMPUTADORAS

Proyecto

Integrantes: Manuel Quiroga – LU: 130621
Germán Stoessel -LU: 118300
Alan Hidalgo – LU:



Versión de Logisim-Evolution: 3.9.0

Etapa 1

Descripción del circuito

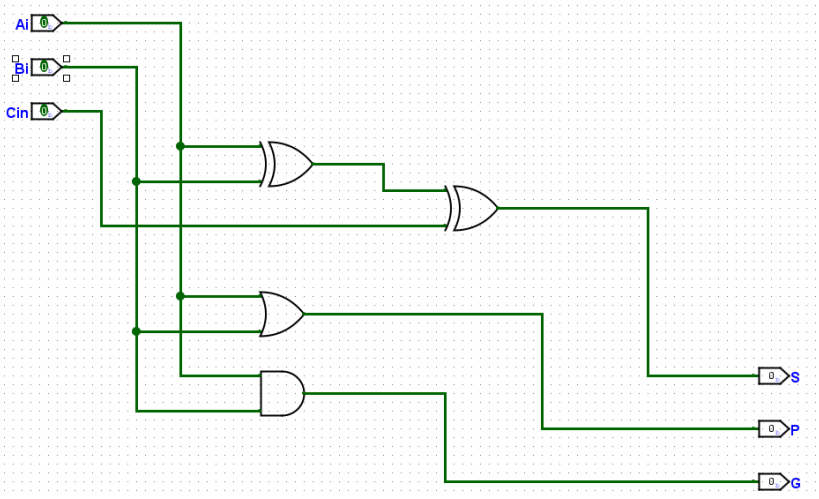
El circuito que armamos en un sumador de 8-bits en el cual utilizamos como base al sumador con acarreo anticipado (CLA: Carry lookahead adder en ingles).

La característica fundamental que tiene esta implementación es que reduce el tiempo del calculo de propagación y generación de los carry's de todos los bits al trabajarlos en manera paralela, a diferencia del Ripple-Adder (Cascada) que debe esperar que el acarreo se propague bit a bit .

Utilizamos un módulo sumador completo o Full-adder FAGP tal que además de obtener las sumas también podamos obtener la propagación y generación de los carry's. A su vez utilizamos otro módulo llamado Bloque_B que se encargará de calcular los carry's de forma anticipada.

FAGP (Full-adder)

Es un sumador full-adder que le añadimos la propagación P y generación G de carry.



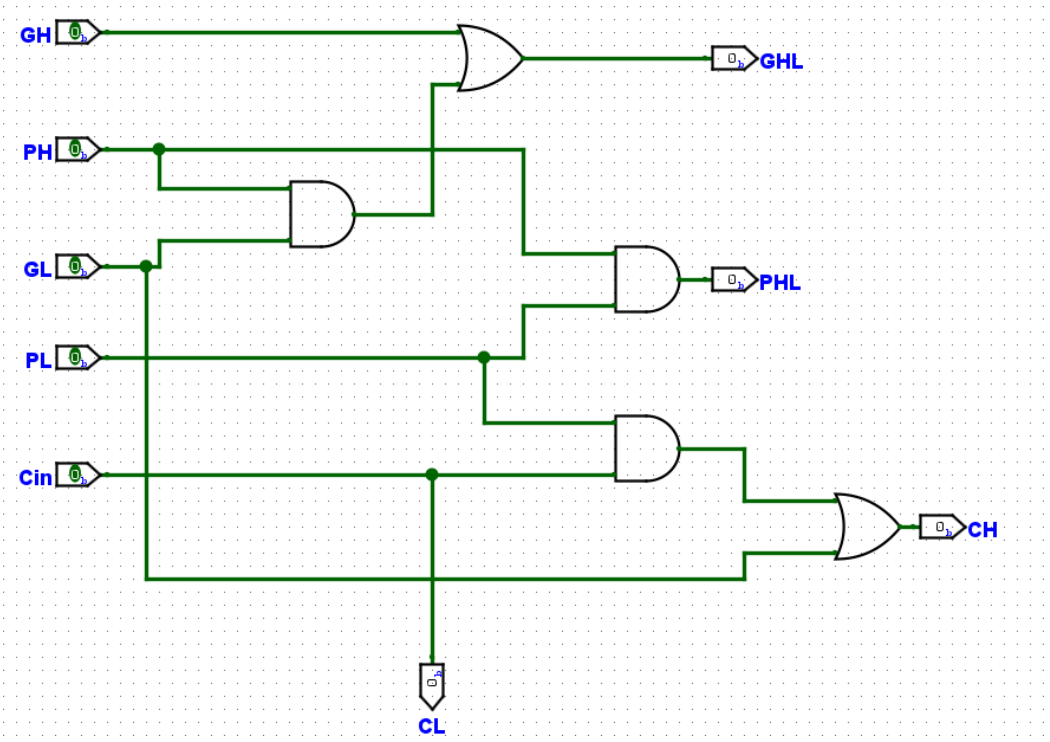
La salida S nos devuelve la suma y para obtenerla se ingresan en una compuerta XOR los bits de cada numero (Ai,Bi), luego al resultado obtenido de esa compuerta viaja hasta otra compuerta XOR junto con el carry de entrada. Entonces la primera suma los bits y la otra aplica el carry correspondiente.

La salida P es la propagación de acarreo, la cual nos devolverá si se propaga o no el acarreo de entrada utilizando una compuerta OR para comparar los bits de Ai y Bi. Si alguno de los dos es 1 se propagaría el acarreo.

La salida G es la generación de acarreo que consta de una compuerta AND donde el único caso que se genere acarreo es cuando ambos bits de entrada sean 1

Bloque B

Este bloque lo utilizamos para calcular los acarreos en paralelo haciendo uso de los acarreos de bloques anteriores.



Como vemos en la imagen, este modulo tiene cinco entradas y cuatro salidas.

En las entradas tenemos dos de ellas que se identifican con la parte menor del bloque b (GL y PL), hay otras dos que son la parte mayor del bloque (GH y PH) y la ultima entrada es el acarreo de entrada (Cin).

El carry-low (CL) viene del acarreo de un bloque anterior, el carry-high (CH) se obtiene a partir de una compuerta AND entre la propagación y el acarreo entrante donde luego ese resultado pasa por una compuerta OR con la generación entrante.

$$CL = C_{in}$$

$$CH = GL + (PL \cdot C_{in})$$

Por otra parte PHL es el carry de propagación total del bloque, obteniéndose a partir de una compuerta AND con las propagaciones entrantes de la parte del bloque menor y del mayor.

$$\boldsymbol{PHL = PL \cdot PH}$$

Y la ultima salida es la que calcula la generación total del bloque (GHL) a partir de la siguiente formula:

$$\boldsymbol{GHL = GH + (GL \cdot PH)}$$

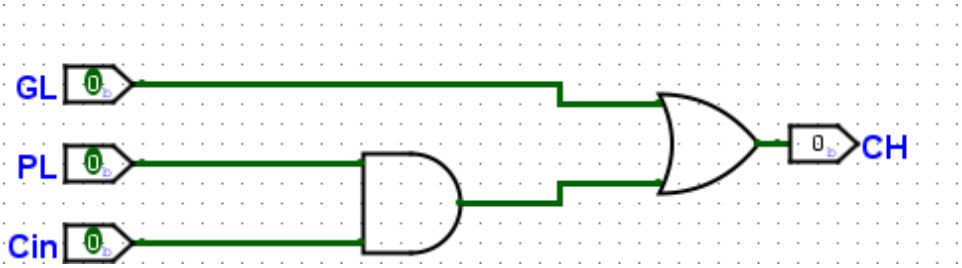
Todo este bloque se aplica utilizando todos los bits de los números A y B para calcular y obtener los acarreos de propagación y generación de manera paralela lo cual acelera el tiempo de los cálculos de los acarreos.

Por un lado los bits menos significativos reciben el acarreo de entrada de manera directa y por otro lado los demás bit van obteniendo sus valores de propagación (PHL) y generación (GHL) que indicarán al siguiente bloque si le envían los acarreos o no.

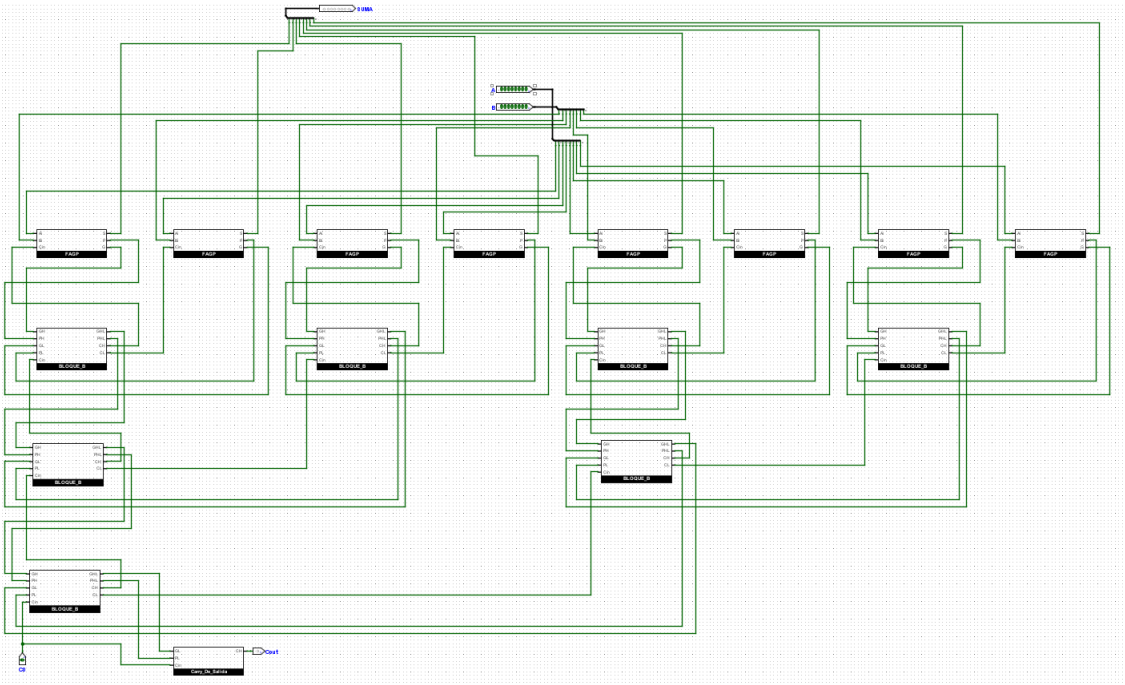
SUMLAT

Es nuestro sumador de 8-bits con acarreo anticipado que buscábamos crear, utilizamos varios bloques b y varios full-adder lo cuales detallamos anteriormente. Le añadimos a su vez un pequeño modulo más para mostrar el acarreo de salida final del circuito el cual nos ayuda a identificar un posible overflow.

La siguiente imagen muestra como planteamos la muestra del carry-out final del circuito.



Finalmente llegamos a completar nuestro sumador, mostramos en la siguiente imagen como quedó el circuito a partir del uso de FAGP y Bloque B.



Etapa 2

Descripción del circuito

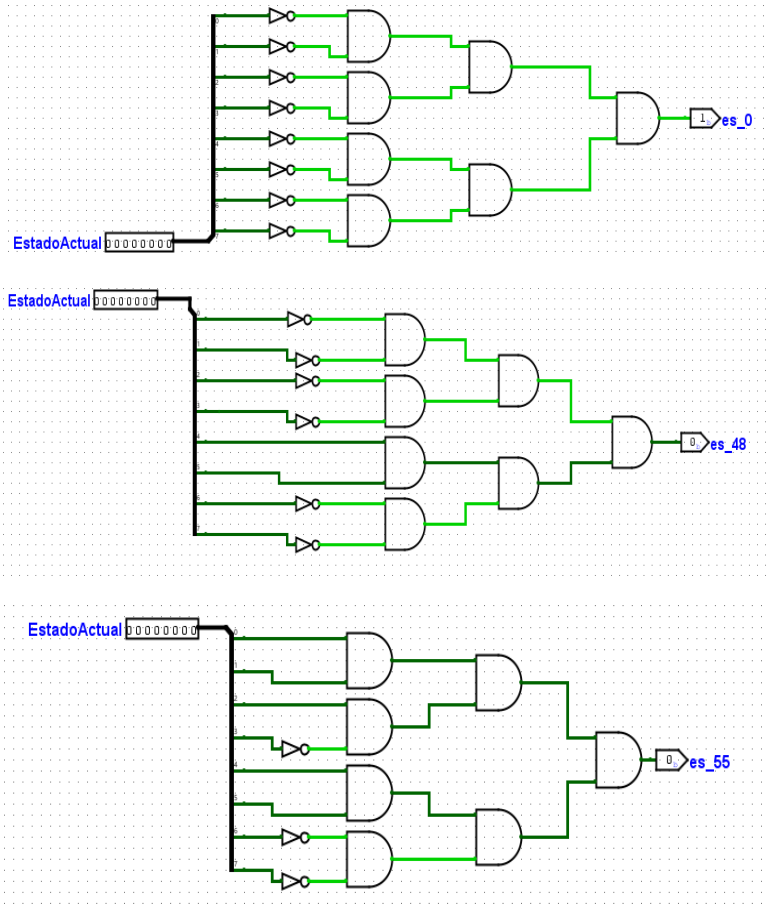
Para la segunda etapa del proyecto teníamos como objetivo hacer un contador de 8 bits en el cual tenía que desempeñarse dependiendo de si estaba en funcionando en modo normal o en un modo alternativo.

Para ello usamos el contador de logisim-evolution sumado a algunas funciones para diferenciar los distintos modos y también contamos con nuestro sumador SUMLAT ya implementado en la etapa 1.

Primero había que plantear como manejar los estados que necesitábamos, como solamente nos importan 6 podíamos descartar el resto porque eran estados que no tendría que llegar el contador.

Comparadores





Como vemos en las siguientes imágenes planteamos los comparadores ajustando al estado deseado, tuvimos que plantear el comparador del estado 0 para que comience el contador dado que no podía arrancar desde el primer estado deseado, pero solo sirve para el arranque y nunca más se llega al estado 0.

El objetivo era que el contador comience desde el estado 48 avance al 49,50,51,52 y en el estado 52 usar el sumador SUMLAT de la etapa 1 para que saltara al estado 55 (52+3), todo esto si nos situábamos en el modo normal.

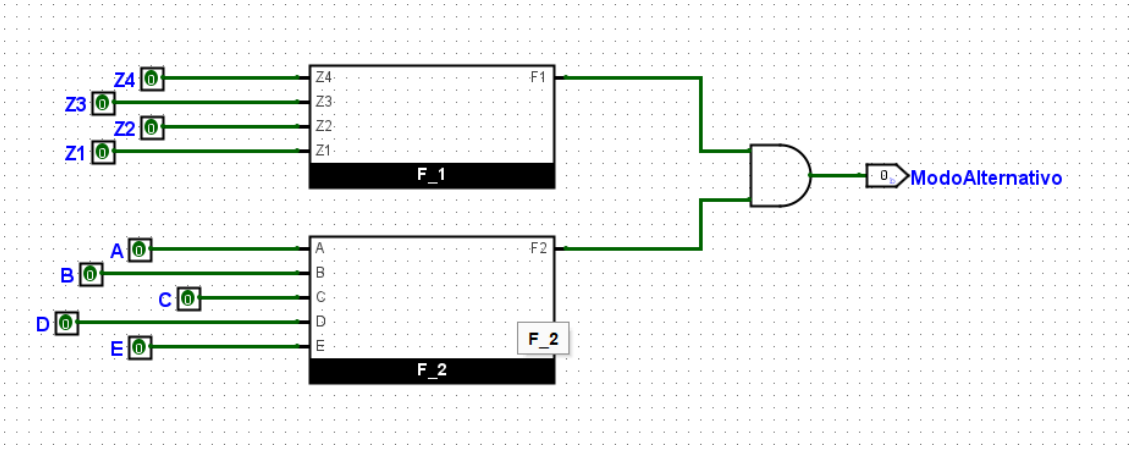
Si estábamos en modo alternativo y nos situábamos en el estado 48 o 49 teníamos que usar el sumador para saltar de estado (del 48 al 51(48+3) y del 49 al 52(49+3)).

Los comparadores de los estados los planteamos dejando la entrada del correspondiente bit que cuando no este negado este encendido (o sea que valga 1) y los bits que en binario corresponden a 0 se niegan para que en caso de valer 1 ya entre en juego la idea del comparador. El ejemplo más notorio es el estado 0.

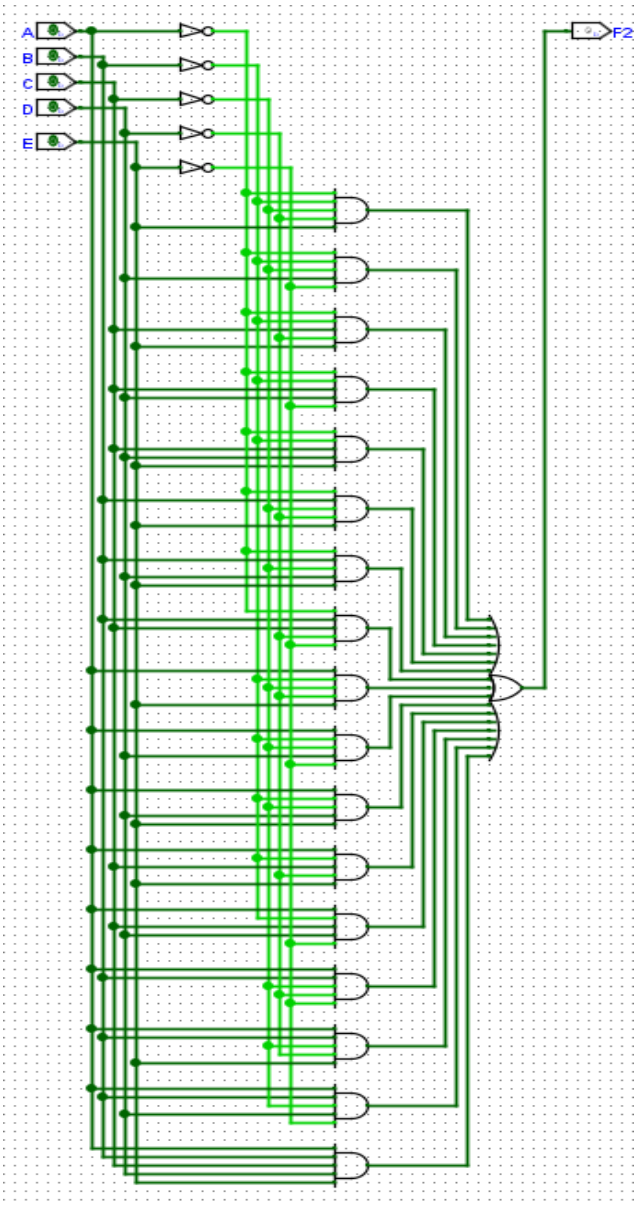
Modo alternativo

Para hacer el modo alternativo había que cumplir con 2 funciones: F1 y F2.

Este modo se enciende cuando el resultado de la compuerta AND da 1 con F1 y F2.



La función F2 fue dada por la Catedra mediante una tabla, la cual solo tuvimos que hacer las conexiones mediante las compuertas necesarias.

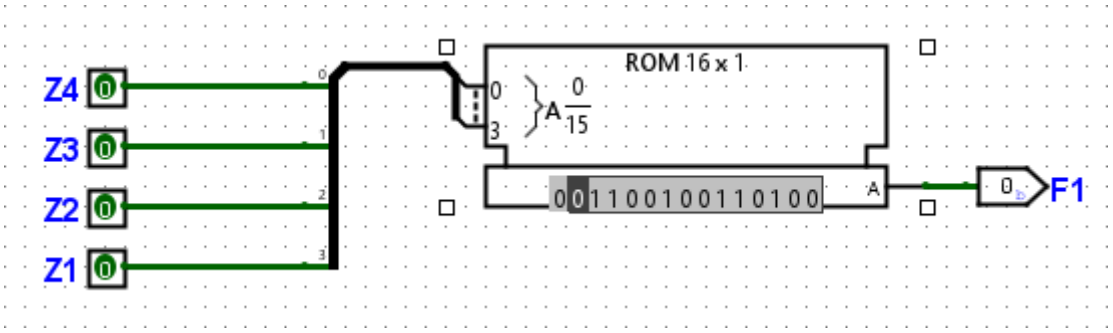


A	B	C	D	E	modoalt
0	0	0	0	0	0
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	1
0	0	1	1	1	1
0	1	0	0	0	0
0	1	0	0	1	1
0	1	0	1	0	0
0	1	0	1	1	1
0	1	1	0	0	1
0	1	1	0	1	0
0	1	1	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	1
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	1
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	0	1	1
1	1	0	1	0	1
1	1	0	1	1	0
1	1	1	0	0	1
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	1

Para la función F1 utilizamos una ROM de logisim de 4 bits de dirección en donde le cargamos los datos de la siguiente tabla en base a la definición de F1 que nos brindaron.

$$F1 = \overline{Z2}\cdot\overline{Z3}\cdot\overline{Z4}+\overline{Z1}\cdot Z2\cdot\overline{Z3}\cdot Z4+Z1\cdot\overline{Z2}\cdot Z4+Z1\cdot Z2\cdot\overline{Z4}+Z1\cdot Z3\cdot Z4$$

ADRESS	DATA
0000	0
0001	1
0010	1
0011	0
0100	0
0101	1
0110	0
0111	0
1000	1
1001	1
1010	0
1011	1
1100	0
1101	0
1110	1
1111	1



Contador 8 bits

Usamos el contador provisto por logisim y lo seteamos para que maneje 8 bits, utilizamos 3 conexiones para darle las instrucciones al contador sobre como actuar en cada caso dependiendo del estado actual en que se encontraba y también dependiendo de en que modo de operación estaba activo.

Usamos las conexiones enable, data y load las cuales:

Enable = activa el conteo normal

Load = carga un valor específico

Data = es el valor a cargar si load = 1.

Mediante compuertas hicimos las conexiones para que enable, load y data funcionaran de manera esperada, a continuación detallaremos como fueron realizadas.

Para activar el conteo normal (de 1 en 1) nos paramos con una compuerta OR para comparar el estado 48 y 49, el resultado de esa compuerta se compara con un AND en la cual entra la señal de que no estamos en modo alternativo. Por otra parte usamos una compuerta OR de 3 entradas en las cuales conectamos la salida del AND anterior y las señales del estado 50 y 51, esto lo hacemos porque así evaluamos sin importar en el modo que me encuentre que se manejan las opciones para llegar al estado deseado. En modo normal se avanza de a 1 pero en caso de estar en modo alternativo los primeros estados (48 y 49) se comportan distinto pero los estados 50, 51 y 52 se compartan igual en los 2 modos.

$$enable = (((48 + 49) * notModoAlternativo) + 50 + 51)$$

Para el load tuvimos que analizar primero en que estado nos situabamos por eso mismo planteamos crear una señal loadAlternativo para que nos indique si estábamos parado en el modo alternativo o no, a continuación lo explicaremos con la siguiente fórmula.

$$LoadAlternativo = (48 + 49) * ModoAlternativo$$

Luego podemos usar esa señal para hacer el load que conectaremos a nuestro contador de la siguiente manera:

Haciendo uso de una compuerta OR de 4 entradas comparamos si el estado actual es el 52 o el 55 o si esta activo el LoadAlternativo o si es el estado 0 (este solo se activa al comienzo del circuito o cada vez que se resetee manualmente)

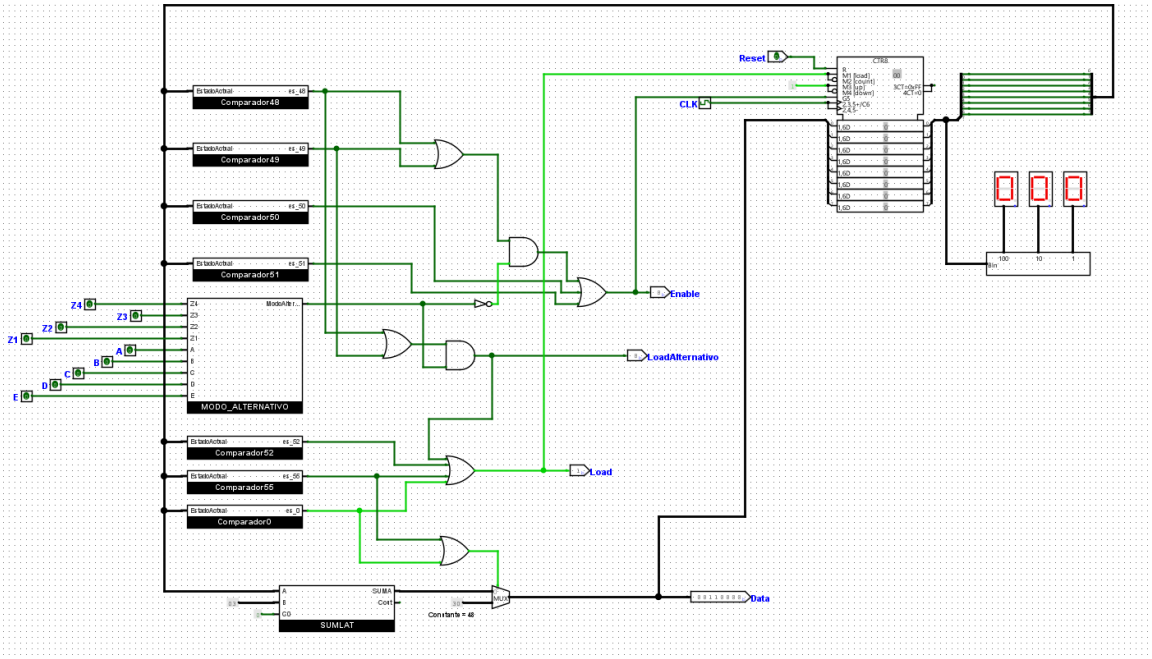
$$Load = LoadAlternativo + 52 + 55 + 0$$

Y por ultimo nos queda la conexión Data la cual nos va a cargar el valor necesario a las entradas D0..D7 del contador cuando la señal Load = 1.

Para hacer esta conexión usamos al sumador de 8 bit SUMLAT y un multiplexor de 2 entradas, en la entrada 0 del multiplexor va la señal del resultado de la suma de SUMLAT esto si el estado es 52 o los estados que en modo alternativo necesiten saltar de a 3 (48 o 49). La entrada 1 del multiplexor pasa la constante 48 que en logisim aparece como 30 porque está en hexadecimal.

El ”select” del multiplexor se une a la señal emitida por una compuerta OR con las señales del estado 0 y el estado 55 que nos van a ayudar a cargar la constante 48 para mostrar el primer estado en caso de estar en el 0 o para pasar al siguiente estado desde el 55.

La señal Data se conecta a las entradas de D0..D7 del contador y va a cambiar sus datos solo cuando Load = 1.



Mostramos la imagen del contador de 8 bit creado y notamos que le añadimos como algo extra unos displays para ver de mejor manera el estado en el estamos parado en cada momento, para ello usamos 3 displays hexadecimal y usamos una función de logisim que nos convierte números de binario a BCD. Solo tuvimos que conectar las salidas de esa conversión y nuestros displays mostraban los estados en base decimal.

Tablas de estados

Modo Normal

Estado actual	Acción	Estado Siguiente	Control que sucede
0	Cargar 48	48	Es_0 = 1 y Load = constante 48
48	Contar +1	49	Es_48 = 1 Enable = 1
49	Contar +1	50	Es_49 = 1 Enable = 1
50	Contar +1	51	Es_50 = 1 Enable = 1
51	Contar +1	52	Es_51 = 1 Enable = 1
52	Cargar 55 (52+3)	55	Es_52 y Load = salida del SUMLAT
55	Cargar 48	48	Es_55 = 1 y Load = constante 48

Modo Alternativo

Estado actual	Acción	Estado Siguiente	Control que sucede
0	Cargar 48	48	Es_0 = 1 y Load = constante 48
48	Cargar 51 (48+3)	51	loadAlternativo =1 y Load = salida del SUMLAT
49	Cargar 52 (49+3)	52	loadAlternativo =1 y Load = salida del SUMLAT
50	Contar +1	51	Es_50 = 1 Enable = 1
51	Contar +1	52	Es_51 = 1 Enable = 1
52	Cargar 55 (52+3)	55	Es_52 y Load = salida del SUMLAT
55	Cargar 48	48	Es_55 = 1 y Load = constante 48

Conclusión

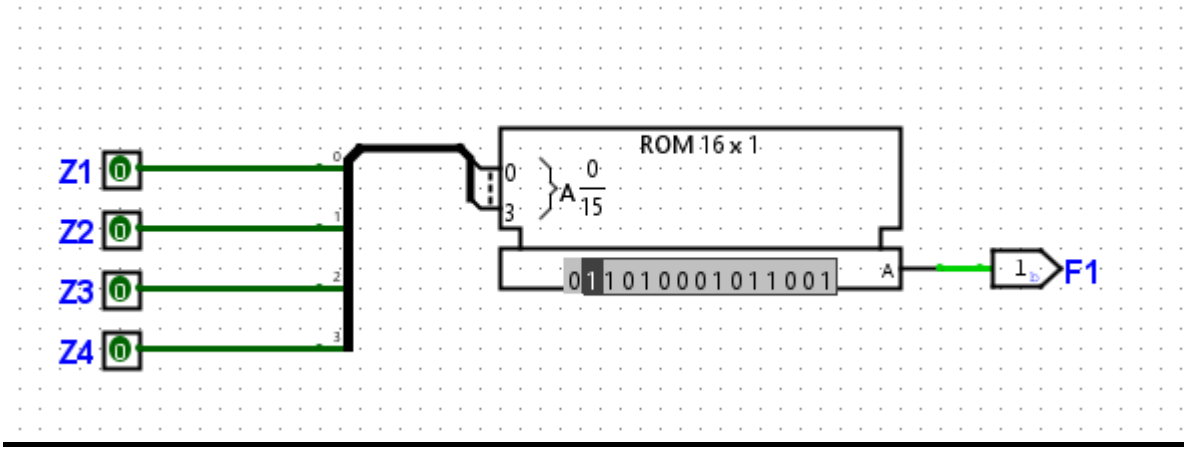
Gracias a este proyecto de Arquitectura de Computadoras pudimos utilizar los conceptos de técnicas digitales y de diseño de circuitos junto con la ayuda de la herramienta Logisim-Evolution.

Incorporar el contador a la primera etapa fue lo que nos desafió a plantear el diseño que utilizamos, desde lo de plantear los comparadores de los estados hasta lograr manejar ambos modos de juego sin complicaciones.

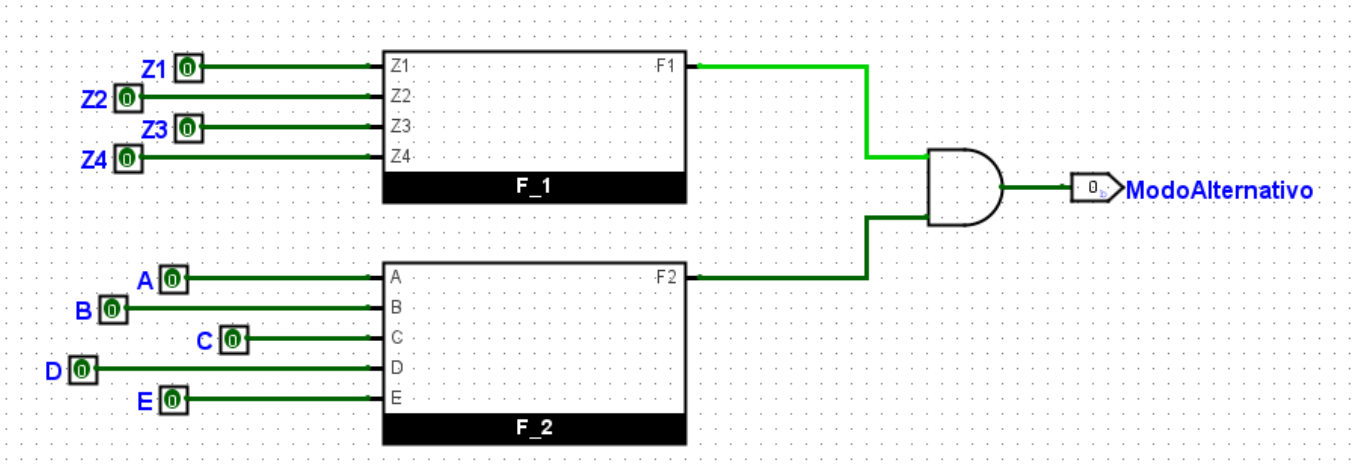
Correcciones

Actualizamos la función F1 que no se activaba correctamente. Modificamos los valores ingresados en la ROM.

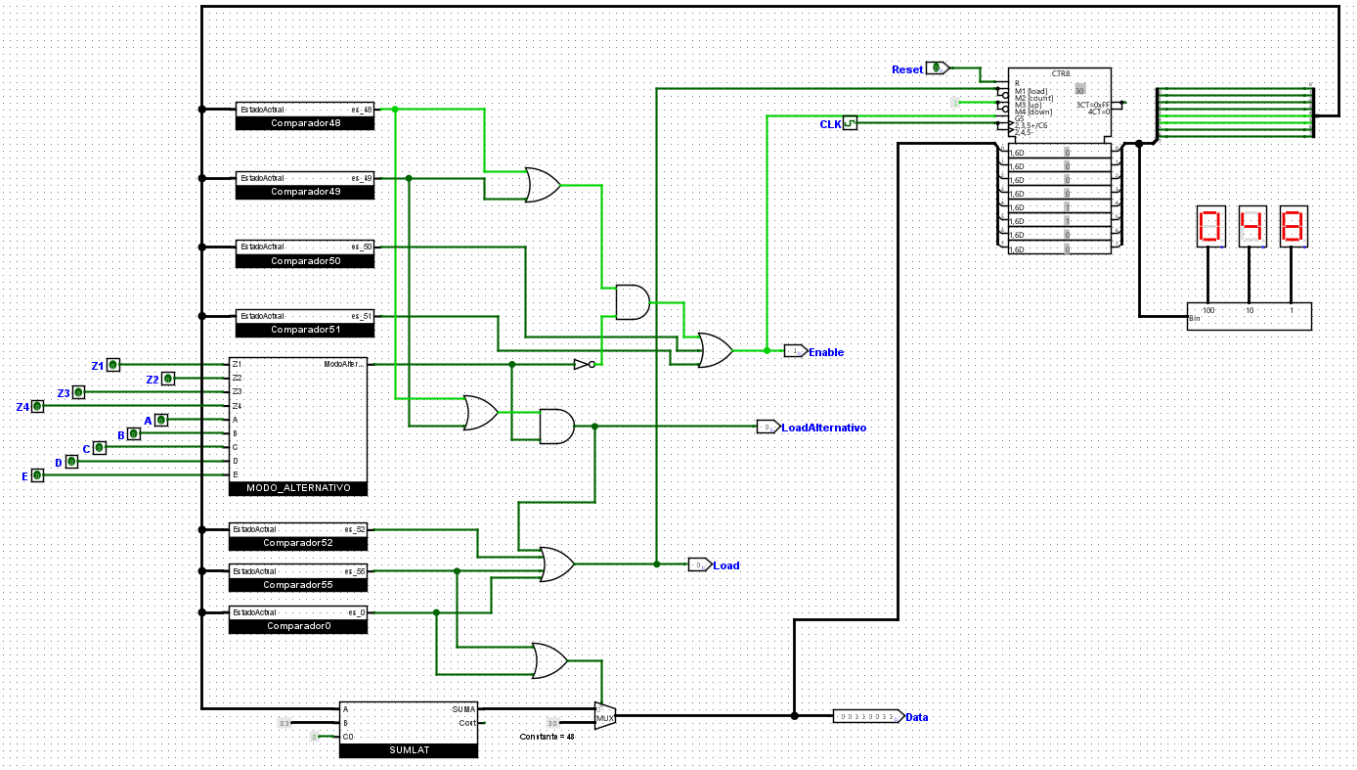
ADRESS	DATA
0000	1
0001	0
0010	0
0011	0
0100	0
0101	1
0110	0
0111	0
1000	1
1001	1
1010	0
1011	1
1100	1
1101	0
1110	1
1111	1



Actualizamos algunas imágenes a continuación donde se reflejan algunos cambios:



Actualización imagen modoAlternativo.



Actualización de la imagen del circuito completo